

深度学习实验报告

专业 智能科学与技术 姓名 史峰源 学号 2412526

课程：深度学习实验 指导教师：李欢 报告日期：2026 年 10 月 4 日

实验一：多层感知机

Fashion-MNIST 分类、泛化分析与 XOR 非线性分类

代码与实验结果：https://github.com/summerwind0131/nku_deep_learning

1 实验目的与准备

本实验以 Fashion-MNIST 分类为任务，实现至少含两个隐藏层的多层感知机（MLP）。首先完成训练曲线、参数量、网络结构、激活函数和损失函数五项基本实验，再研究模型复杂度、过拟合、L2、Dropout、Batch Normalization，并使用神经网络解决 XOR 问题。

Fashion-MNIST 图像大小为 28×28 ，共有 10 类。使用 `ToTensor()` 将像素缩放至 $[0, 1]$ ，进入网络时展平为 784 维向量，不使用额外的数据增强。将官方 60,000 张训练图像按类别分层划分为 50,000 张训练图像和 10,000 张验证图像；官方 10,000 张测试图像保持独立。划分种子固定为 42，各组共用同一划分。

表 1 Fashion-MNIST 实验的共同设置

设置项	取值	设置项	取值
优化器	Adam	学习率	10^{-3}
批量大小	64	训练随机种子	42
基础对比轮数	20	大模型对比轮数	60
基线隐藏层	256-128	基线激活 / 损失	ReLU / 交叉熵
优化器 weight decay	0	学习率调度	无

实验使用 RTX Laptop GPU，Python 3.10.20、PyTorch 2.11.0+cu128。每轮保存模型并计算验证误差，以验证误差最低的轮次保存 `best.pth`；测试曲线由训练结束后逐轮评估已保存模型得到。下文“最佳轮”均指验证误差最低的轮次，“最佳轮测试误差”为该轮模型在测试集上的误差。

2 基本实验要求

2.1 要求 1：绘制训练损失、训练误差和测试误差曲线

实现方法。基线网络结构为 $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ ，两个隐藏层使用 ReLU，输出层给出 10 类的 logits。交叉熵直接接收 logits，在内部完成与 Softmax 对应的计算，因此不在模型输出处另加 Softmax。关键训练步骤如下：

```
model = MLP(input_dim=784, hidden_dims=[256, 128],
            output_dim=10, activation="relu")
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001,
                              weight_decay=0.0)

model.train()
for images, labels in train_loader:
    optimizer.zero_grad()
    logits = model(images)
    loss = criterion(logits, labels)
    loss.backward()
    optimizer.step()
```

每轮训练损失按各批次样本数加权平均，分类误差则在该轮结束后，切换至 `model.eval()` 并关闭梯度，遍历数据集计算：

$$\bar{L}_{\text{train}} = \frac{\sum_b n_b L_b}{\sum_b n_b}, \quad \text{err} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left[\arg \max_k z_{ik} \neq y_i \right]. \quad (1)$$

这里 L_b 是当前批次的平均损失， n_b 是其样本数。训练损失反映一轮内不断更新的模型，训练误差和测试误差反映轮末模型。

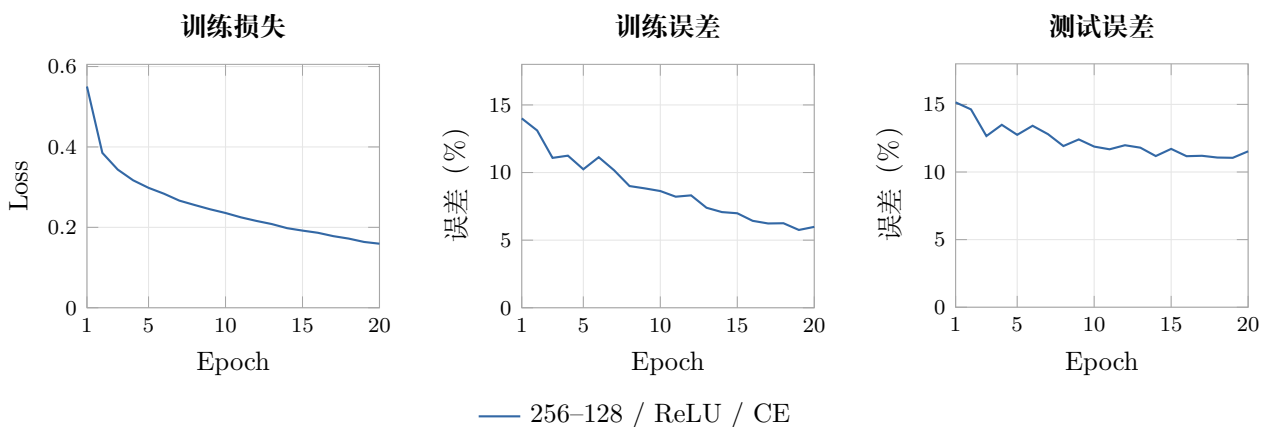


图 1 基线网络的训练损失、训练误差和测试误差。横轴均为训练轮次。

结果与分析。图1中，训练损失从第 1 轮的 0.5504 降至第 20 轮的 0.1593，训练误差从 14.00% 降至 5.98%，测试误差从 15.15% 降至 11.53%。前期两种误差同时下降，说明模型学到了能够用于未见图像的特征；后期训练误差继续减小，测试误差下降趋缓并有波动，说明继续拟合训练数据未必带来同等幅度的泛化改善。验证集选出的最佳轮次为第 19 轮，其验证误差为 9.90%，测试误差为 11.05%。因此，仅凭训练损失仍在下降，不能判断继续训练一定能改善测试表现。

2.2 要求 2：计算网络参数量，包括权重数和偏置数

计算方法。输入维数为 d_{in} 、输出维数为 d_{out} 的全连接层含 $d_{in}d_{out}$ 个权重和 d_{out} 个偏置。因此，基线网络的参数量为

$$P = (784 + 1) \times 256 + (256 + 1) \times 128 + (128 + 1) \times 10 = 235,146. \quad (2)$$

表 2 基线网络参数量的逐层计算

全连接层	权重数	偏置数	合计
784 → 256	200,704	256	200,960
256 → 128	32,768	128	32,896
128 → 10	1,280	10	1,290
总计	234,752	394	235,146

通过代码对模型参数进行核对：

```
num_params = sum(p.numel() for p in model.parameters())
```

结果与分析。程序统计与手算一致。Flatten 和 ReLU 没有可学习参数；第一层全连接占基线参数量约 85.5%，因此调整第一隐藏层宽度对总参数量影响尤其明显。

2.3 要求 3：改变隐藏层层数或神经元数，比较曲线与参数量

实验设计与核心代码。固定 ReLU、交叉熵及表 1 的训练设置，比较 64–32、256–128、512–256 三种两隐藏层结构，并加入 256–128–64 三隐藏层结构。每组均训练 20 轮。网络根据隐藏层维度列表逐层构造：

```
layers = [nn.Flatten(start_dim=1)]
previous_dim = input_dim
for hidden_dim in hidden_dims:
    layers.append(nn.Linear(previous_dim, hidden_dim))
    layers.append(activation_class())
    previous_dim = hidden_dim
layers.append(nn.Linear(previous_dim, output_dim))
self.network = nn.Sequential(*layers)
```

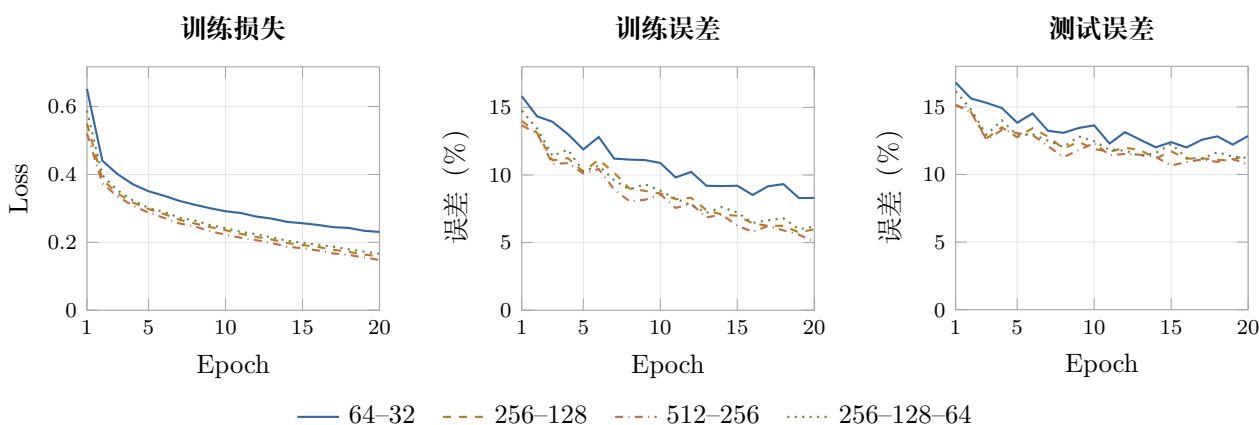


图 2 不同隐藏层结构下的三类训练曲线，训练预算均为 20 轮。

表 3 结构对比：参数量与验证集选中轮次的结果

隐藏层结构	参数量	最佳轮	验证误差	对应测试误差
64-32	52,650	16	10.62%	12.00%
256-128	235,146	19	9.90%	11.05%
512-256	535,818	20	9.48%	10.73%
256-128-64	242,762	13	9.76%	11.36%

结果与分析。在两隐藏层网络中，增大宽度带来了更强的拟合能力。第 20 轮时，64–32、256–128、512–256 的训练误差分别为 8.31%、5.98% 和 5.06%，测试误差分别为 12.85%、11.53% 和 10.73%。结合曲线可以看到，窄网络的训练损失下降后停留在较高水平，宽网络在本次运行中同时改善了训练和测试表现。

增加一个 64 维隐藏层后，参数量从 235,146 增至 242,762。其最佳验证误差为 9.76%，略低于基线，但对应测试误差 11.36% 略高于基线的 11.05%。这说明增加深度并没有在本实验中带来一致改善。深度增加改变了特征变换方式，也增加了优化难度；它与简单增加宽度的效果不能等同。这里同时改变了结构和参数量，不能把差异全部归因于“层数”这一个因素。

2.4 要求 4：尝试不同激活函数，观察三类曲线

实验设计与核心代码。保持隐藏层为 256-128，分别在两个隐藏层使用 ReLU、Tanh 和 Sigmoid，其他设置完全相同。三组参数量均为 235,146。

```
activation_classes = {
    "relu": nn.ReLU,
    "tanh": nn.Tanh,
    "sigmoid": nn.Sigmoid,
}
activation_class = activation_classes[activation]
layers.append(activation_class())
```

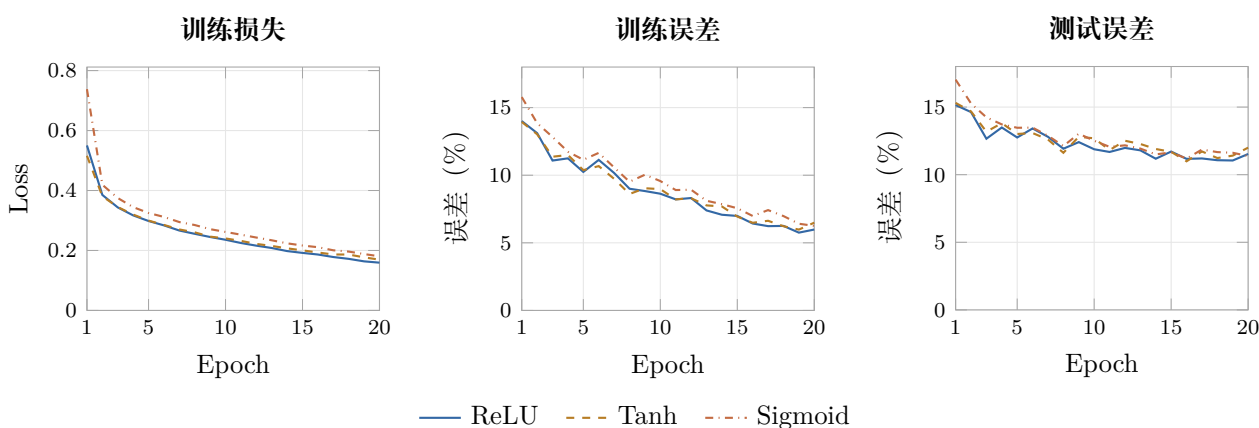


图 3 不同激活函数下的训练损失、训练误差和测试误差。

表 4 激活函数对比：验证集选中轮次的结果

激活函数	最佳轮	验证误差	对应测试误差
ReLU	19	9.90%	11.05%
Tanh	19	9.98%	11.41%
Sigmoid	16	10.04%	11.19%

结果与分析。第一轮训练损失分别为 ReLU 0.5504、Tanh 0.5159、Sigmoid 0.7380，Sigmoid 的前期学习较慢。Sigmoid 的导数最大为 1/4，且在输入绝对值较大时接近零；Tanh 也有饱和区，而 ReLU

在正区间保持导数为 1。这些性质可以解释激活函数可能对梯度传播产生的影响，但本实验未记录逐层梯度，不能仅凭曲线确认具体的梯度消失程度。

经过 20 轮训练，三种激活均可获得较好的分类结果，最佳验证误差的最大差距只有 0.14 个百分点。ReLU 在本次实验的最佳模型测试误差最低，但优势较小；对于当前两隐藏层网络，激活函数改变主要影响早期学习速度，最终分类误差并未拉开很大差距。

2.5 要求 5：尝试不同损失函数，观察三类曲线

实验设计与核心代码。固定 256-128、ReLU，对比交叉熵和均方误差。对于 MSE，先将 logits 转换为类别概率，将标签转换为 one-hot 向量，再计算均方误差。若类别数 $K = 10$ ，则两种目标为

$$L_{\text{CE}} = -\frac{1}{N} \sum_i \log p_{i,y_i}, \quad L_{\text{MSE}} = \frac{1}{NK} \sum_i \sum_k (p_{ik} - \mathbb{I}[y_i = k])^2. \quad (3)$$

```
class ClassificationMSELoss(nn.Module):
    def forward(self, logits, labels):
        probabilities = F.softmax(logits, dim=1)
        targets = F.one_hot(labels, num_classes=logits.shape[1])
        targets = targets.to(dtype=logits.dtype)
        return F.mse_loss(probabilities, targets)

def build_loss(name):
    if name == "cross_entropy":
        return nn.CrossEntropyLoss()
    if name == "mse":
        return ClassificationMSELoss()
```

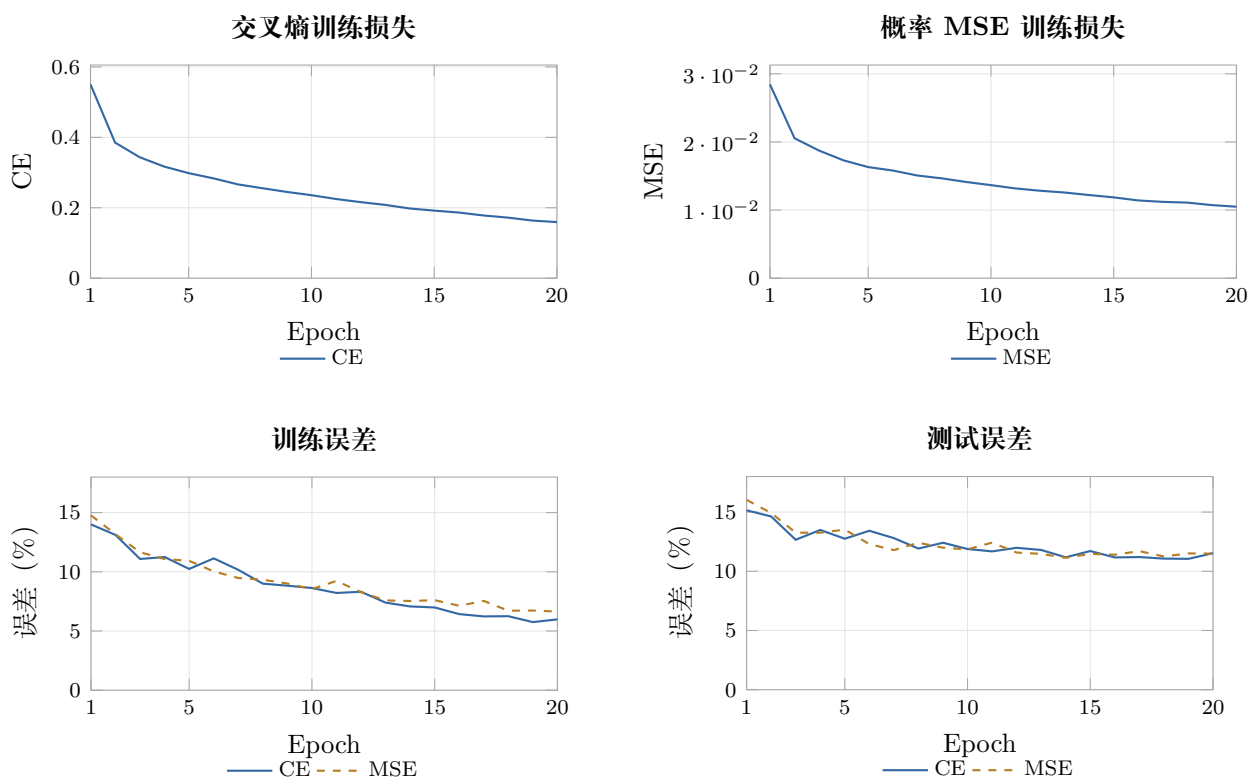


图 4 交叉熵与概率 MSE 的对比。上排分别展示训练损失，下排用相同误差尺度比较分类表现。

表 5 损失函数对比：验证集选中轮次的结果

损失函数	最佳轮	验证误差	对应测试误差
交叉熵 CE	19	9.90%	11.05%
Softmax + one-hot MSE	18	9.76%	11.25%

结果与分析。两种损失都能有效训练网络，最佳模型的测试误差均约为 11%。MSE 的最佳验证误差略低，但其对应测试误差为 11.25%，略高于交叉熵的 11.05%。因此，本次结果没有显示 MSE 带来明确的分类优势。

MSE 曲线的数值明显小于交叉熵，不能据此认定它训练得更好：两者定义和尺度不同，MSE 还对 10 个类别取了平均。交叉熵在真实类别概率接近零时惩罚很大，而概率 MSE 有界，两者对错误预测的梯度反馈也不同。公平判断应结合各自损失的下降趋势以及共同的分类误差指标。此处使用相同学习率，未分别为两种损失调优。

3 附加题

3.1 附加题 1：模型复杂度与训练误差、测试误差的关系

在结构实验基础上，增加隐藏层为 1024-1024-512 的大模型。实现仍使用前述 `hidden_dims` 循环，只需改变列表即可。为避免把训练轮数的影响误认为模型复杂度的影响，先将五种结构统一比较到第 20 轮：

表 6 固定第 20 轮时，不同复杂度模型的分类型误差

隐藏层结构	参数量	训练误差	测试误差
64-32	52,650	8.31%	12.85%
256-128	235,146	5.98%	11.53%
512-256	535,818	5.06%	10.73%
256-128-64	242,762	6.18%	11.22%
1024-1024-512	2,383,370	5.67%	11.45%

分析。从 64-32 增宽到 512-256 时，训练和测试误差同时下降，表明较小模型的表达能力限制了结果。但继续扩大到 2,383,370 个参数，在第 20 轮并没有超过 512-256 的测试表现；训练误差也未严格随参数量单调下降。参数量影响可表示的函数范围，实际训练结果还取决于优化路径、随机初始化和训练预算。

大模型训练至 60 轮后，验证集选出的第 56 轮模型取得 9.94% 的测试误差，说明该模型在更长训练下仍有改善空间。但这一结果同时改变了结构和训练轮数，不能作为“参数越多越好”的证据。模型复杂度应结合训练误差是否过高、训练与测试之间的差距及验证表现共同判断。

3.2 附加题 2：观察过拟合；必要时增大模型使其明显

实验方法。采用 1024-1024-512、ReLU、交叉熵，不启用 L2、Dropout 或 BN，训练 60 轮。使用现有参数接口即可完成这一设置：

```
python train.py --name night01_large_plain \
  --hidden-dims 1024 1024 512 --epochs 60
```

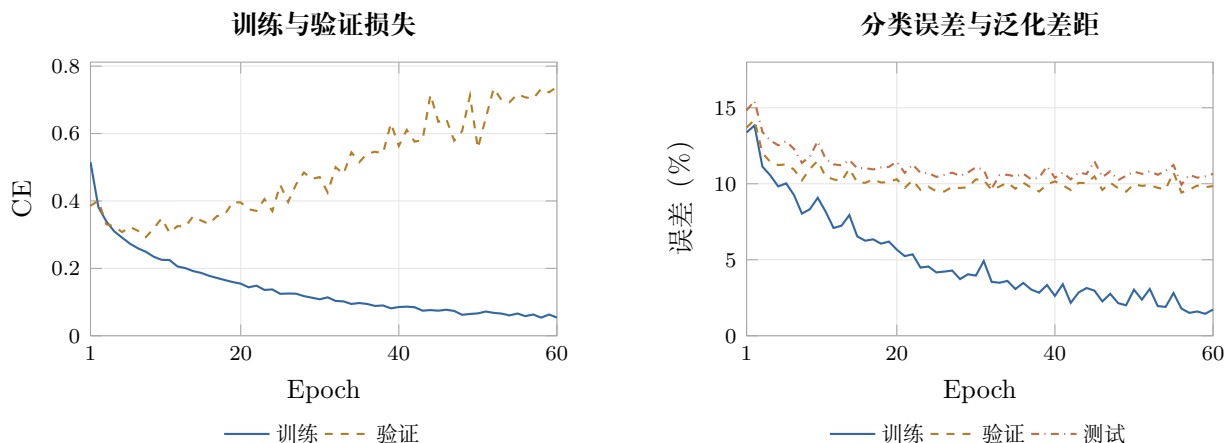


图 5 大模型的过拟合现象：训练和验证损失，以及训练、验证、测试误差。

结果与分析。第 60 轮训练误差降至 1.72%，测试误差仍为 10.65%，两者相差约 8.93 个百分点。与此同时，验证交叉熵在第 8 轮达到约 0.293 后总体升高，到第 60 轮增至约 0.738，而训练损失继续下降至 0.0542。模型对训练数据的拟合越来越好，却没有在未参与训练的数据上获得相应收益，过拟合现象比小模型更明显。

需要区分图中的两个现象：测试分类误差没有持续单调上升，后期部分轮次仍有改善；验证损失却明显恶化。分类误差只关注最大概率对应的类别是否正确，交叉熵还关注真实类别的概率。少量置信度很高的错误预测就可能显著增大交叉熵，而不会同比例增加错误样本数。因此，本组结果应表述为“训练与测试差距增大、验证损失恶化”，而不能笼统声称“越训练测试误差越高”。

3.3 附加题 3：在目标函数中加入 L2 正则化，观察误差

实验设计与核心代码。在同一大模型上比较 $\lambda = 0, 10^{-4}, 10^{-3}$ ，每组均训练 60 轮。将全连接层权重的平方和加入目标：

$$J = L_{\text{data}} + \frac{\lambda}{2} \sum_{\ell} \|W^{(\ell)}\|_F^2. \quad (4)$$

该项向权重梯度中加入 $\lambda W^{(\ell)}$ ，抑制权重过大。本实验不惩罚偏置，也不对 BN 参数施加此项。

```
linear_weights = [layer.weight for layer in model.modules()
                  if isinstance(layer, nn.Linear)]
data_loss = criterion(logits, labels)
total_loss = data_loss
if l2 > 0:
    weight_square_sum = sum(w.square().sum() for w in linear_weights)
    total_loss = data_loss + 0.5 * l2 * weight_square_sum
total_loss.backward()
optimizer.step()
```

优化器的 `weight_decay` 保持为 0，避免重复施加惩罚。日志分别记录数据损失和含 L2 的总目标；下图使用数据损失，验证和测试也只计算数据损失。

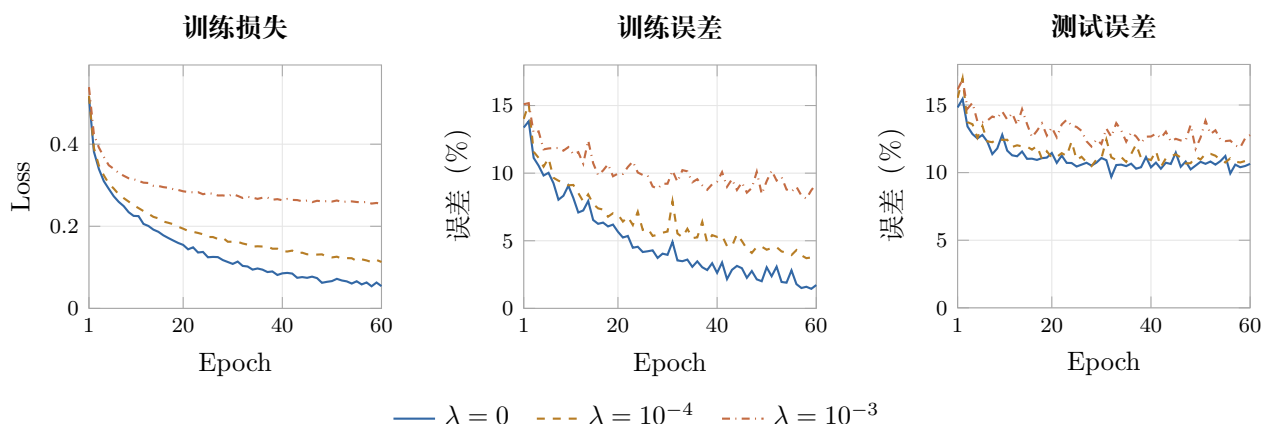


图 6 L2 系数对训练数据损失、训练误差和测试误差的影响。

表 7 L2 对比：固定轮次与验证选中模型的结果

L2 系数	第 60 轮训练误差	第 60 轮测试误差	最佳轮测试误差
$\lambda = 0$	1.72%	10.65%	9.94%
$\lambda = 10^{-4}$	3.88%	10.87%	11.16%
$\lambda = 10^{-3}$	9.21%	12.78%	12.01%

结果与分析。加入 L2 后，训练损失和训练误差保持在更高水平，且 10^{-3} 的影响明显强于 10^{-4} 。 $\lambda = 10^{-3}$ 时，第 60 轮训练误差为 9.21%，测试误差为 12.78%，相较无正则化的 1.72% 和 10.65% 均变差，说明在当前配置下约束过强，限制了有效拟合。 10^{-4} 的约束较温和，但最佳模型测试误差仍未优于无正则化组。L2 通过限制权重降低过拟合风险，并不保证任意系数都会提升泛化；其强度需要通过验证集选择。

3.4 附加题 4：使用 Dropout，观察泛化表现

实验设计与核心代码。在大模型每个隐藏层的激活之后加入 $p = 0.2$ 的 Dropout。训练时随机将 20% 的激活置零，其余激活乘以 $1/(1 - p)$ ；评估时关闭随机丢弃。

```
layers.append(nn.Linear(previous_dim, hidden_dim))
layers.append(activation_class())
if dropout > 0.0:
    layers.append(nn.Dropout(p=dropout))

model.train() # enable dropout while optimizing
model.eval() # disable dropout while evaluating
```

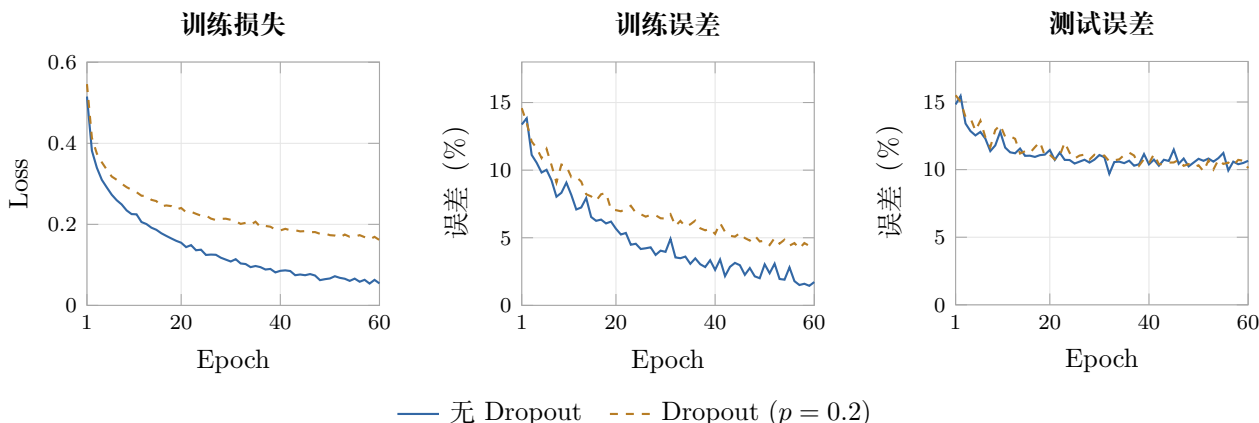


图 7 Dropout 对比。训练损失在启用 Dropout 时记录；训练误差和测试误差均在评估模式下计算。

表 8 Dropout 对比结果

设置	第 60 轮训练误差	第 60 轮测试误差	最佳轮测试误差
无 Dropout	1.72%	10.65%	9.94%
Dropout ($p = 0.2$)	4.29%	10.11%	9.88%

结果与分析。 Dropout 不增加可学习参数，两组均为 2,383,370 个参数。加入 Dropout 后，训练损失下降较慢，最终训练误差由 1.72% 增至 4.29%，测试误差由 10.65% 降至 10.11%；训练与测试的差距由 8.93 缩小至 5.82 个百分点。随机丢弃抑制了模型对部分隐藏单元的过度依赖，本次实验中较弱的训练拟合对应了略好的末轮测试表现。

按验证误差选择模型时，Dropout 组测试误差为 9.88%，无正则化组为 9.94%，只相差 6 个测试样本。该结果支持“本次运行中 Dropout 有一定改善”，还不足以说明其优势稳定或显著。

3.5 附加题 5：使用 Batch Normalization，观察误差

实验设计与核心代码。 在大模型的每个隐藏层中，将 BN 放在全连接层之后、ReLU 之前。对每个特征，训练时按批次计算

$$\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y = \gamma \hat{x} + \beta. \quad (5)$$

```
layers.append(nn.Linear(previous_dim, hidden_dim))
if batch_norm:
    layers.append(nn.BatchNorm1d(hidden_dim))
layers.append(activation_class())
```

训练阶段使用批次统计量并更新运行均值、方差，评估阶段使用保存的运行统计量。因此，评估前的 `model.eval()` 对 BN 同样必要。BN 的可学习参数是每个特征的缩放 γ 与平移 β ，新增 $2 \times (1024 + 1024 + 512) = 5,120$ 个参数，总计 2,388,490；运行均值和方差属于缓冲量，不计入可学习参数。

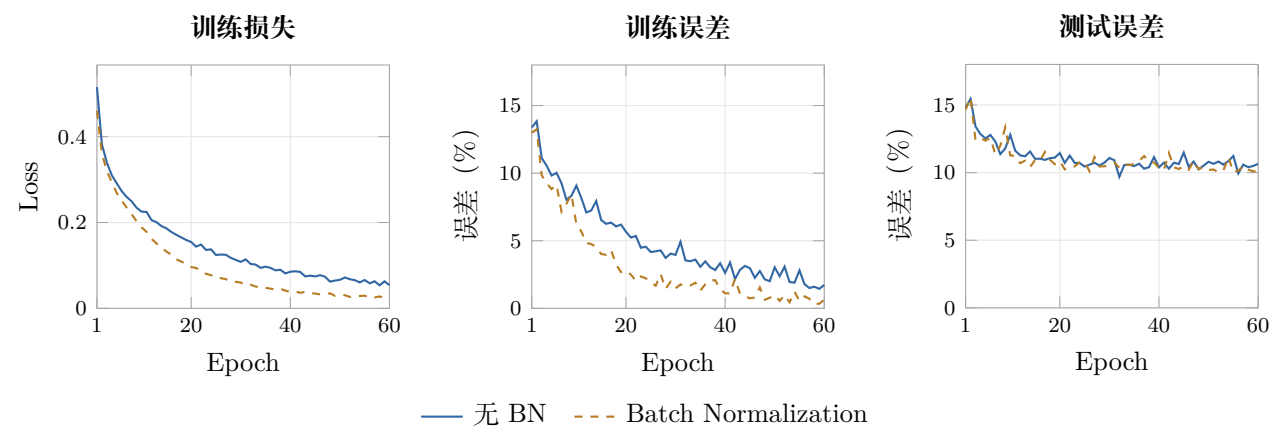


图 8 BN 对训练损失、训练误差和测试误差的影响。

表 9 Batch Normalization 对比结果

设置	第 60 轮训练误差	第 60 轮测试误差	最佳轮测试误差
无 BN	1.72%	10.65%	9.94%
Batch Normalization	0.64%	10.10%	10.03%

结果与分析。BN 组第 60 轮训练误差为 0.64%，低于无 BN 组的 1.72%，测试误差为 10.10%，也低于无 BN 组的 10.65%。它在本次运行中提高了训练拟合程度，并改善了末轮测试表现，作用方式与 L2、Dropout 限制拟合的现象不同。

BN 组在第 52 轮取得最低验证误差 8.98%，对应测试误差为 10.03%。若比较各组验证选出的模型，无 BN 组的测试误差 9.94% 反而略低。这与“第 60 轮 BN 更好”并不矛盾，因为比较的模型轮次不同。BN 也没有消除训练与测试之间的差距，因此不能把归一化等同于解决过拟合。

3.6 附加题 6：训练神经网络解决 XOR 问题

实验设计与核心代码。XOR 在两个输入相同时输出 0，不同时输出 1。四个点不能由一条直线分开，单个线性分类器无法完成该任务。按照课件示例构建 $2 \rightarrow 5 \rightarrow 1$ 网络，隐藏层使用 ReLU，输出层为线性层，采用 MSE 和 SGD；随机种子为 1，学习率为 0.1，全批量更新 5,000 次。

```
class Model(nn.Module):
    def __init__(self):
        super().__init__()
        self.lc1 = nn.Linear(2, 5)
        self.lc2 = nn.Linear(5, 1)

    def forward(self, x):
        return self.lc2(F.relu(self.lc1(x)))

x = torch.tensor([[0., 0.], [0., 1.], [1., 0.], [1., 1.]])
y = torch.tensor([[0.], [1.], [1.], [0.]])
model = Model()
criterion = nn.MSELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
for i in range(5000):
    optimizer.zero_grad()
    loss = criterion(model(x), y)
    loss.backward()
    optimizer.step()
```

网络共有 $(2 + 1) \times 5 + (5 + 1) \times 1 = 21$ 个参数。以输出大于 0.5 判为类别 1，结果如下：

表 10 XOR 四种输入的训练结果

x_1	x_2	目标值	网络输出	判定类别
0	0	0	0.000000517	0
0	1	1	0.999999285	1
1	0	1	0.999999523	1
1	1	0	0.000000670	0

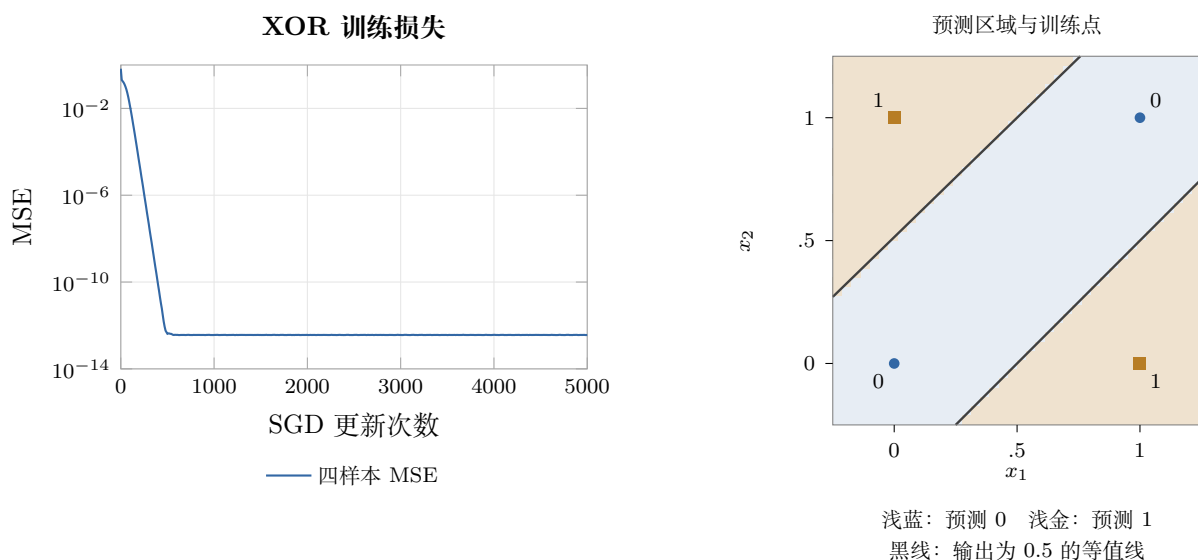


图 9 XOR 的训练损失与预测区域。损失纵轴采用对数刻度；右图黑线为网络输出等于 0.5 的边界。

结果与分析。 MSE 从训练前的 0.6723 降至约 3.64×10^{-13} ，四个输入均分类正确。ReLU 使网络能够表示分段线性函数，经多个隐藏单元组合形成非线性分类边界，从而区分 XOR 的两组对角顶点。若去掉隐藏层非线性，多层线性变换的复合仍是线性变换，无法解决这一问题。这里验证的是四个真值表输入的拟合能力；右图其他位置用于展示网络学到的函数形状，并不代表额外测试样本的正确标签。输出层为线性层，数值也不应解释为概率。

4 实验总结

本实验完成了 MLP 在 Fashion-MNIST 上的分类训练，并围绕课件要求逐项比较了结构、激活函数、损失函数和正则化方法，主要结论如下。

1. **结构影响表达能力，但参数更多不保证测试误差更低。**适度增宽改善了本次基础实验结果；增加深度或进一步扩大参数量，没有在相同 20 轮预算下持续带来收益。
2. **激活函数和损失函数影响优化过程。**Sigmoid 前期学习较慢，但三种激活的最终误差接近；交叉熵与概率 MSE 均可用于训练，比较时应关注共同的分类误差，不能比较原始损失值大小。
3. **训练损失下降不等于泛化持续改善。**大模型后期训练拟合增强，验证损失却恶化。分类误差与交叉熵反映不同性质，应结合观察，并使用独立验证集选择模型。
4. **不同方法改变学习过程的方式不同。**本次 L2 系数较大时约束过强；Dropout 减弱训练拟合并缩小训练与测试的差距；BN 增强了训练拟合、改善了末轮测试表现，但并未消除泛化差距。
5. **隐藏层非线性使网络能够解决线性不可分问题。**小型 ReLU 网络用 21 个参数完成 XOR 真值表的拟合，说明非线性变换的必要性。

本次各配置仅运行一个随机种子，小幅差异可能来自训练随机性；后续可针对基线、Dropout 和 BN 等代表性配置增加多个种子，报告均值与标准差，并进一步在验证集上选择正则化强度。

5 实验代码架构简介

为支持上述控制变量实验，代码将参数配置、数据处理、模型、训练和评估分开。结构、激活、损失、L2、Dropout 与 BN 均由参数指定，使不同实验复用相同训练流程。

表 11 实验代码的主要职责划分

文件	职责
config.py、data.py	解析实验参数；加载数据并固定训练、验证划分。
model.py、losses.py	构造不同隐藏层结构、激活、Dropout、BN 以及分类损失。
engine.py、train.py	单轮训练、误差计算；组织训练并保存日志和检查点。
run_experiments.py	按预配置顺序运行各组实验，减少手工修改和重复启动。
evaluate.py、plot.py	评估已保存模型，整理误差和损失曲线。
XOR.py	独立实现 XOR 网络及其训练。

每个实验单独保存配置与 CSV 指标，last.pth 用于恢复训练，best.pth 保存验证误差最低的模型。恢复时同时加载模型、优化器和随机状态，使中断后的训练可以继续；W&B 使用离线模式记录同一组指标。以上组织方式主要服务于实验的可重复运行、结果比较和检查点复用。